

Python String Methods

1. **capitalize()** - Converts the first character of the string to uppercase and the rest to lowercase.
2. **casefold()** - Converts the string to lowercase (more aggressive than lower()).
3. **center(width[, fillchar])** - Centers the string with padding.
4. **count(sub[, start[, end]])** - Returns the count of occurrences of a substring.
5. **encode(encoding[, errors])** - Encodes the string using the specified encoding.
6. **endswith(suffix[, start[, end]])** - Checks if the string ends with the specified suffix.
7. **expandtabs(tabsize=8)** - Replaces tabs with spaces.
8. **find(sub[, start[, end]])** - Finds the first occurrence of a substring; returns -1 if not found.
9. **format(*args, **kwargs)** - Formats the string using placeholders.
10. **format_map(mapping)** - Formats the string using a dictionary or mapping.
11. **index(sub[, start[, end]])** - Finds the first occurrence of a substring; raises ValueError if not found.
12. **isalnum()** - Checks if all characters are alphanumeric.
13. **isalpha()** - Checks if all characters are alphabetic.
14. **isascii()** - Checks if all characters are ASCII.
15. **isdecimal()** - Checks if all characters are decimal digits.
16. **isdigit()** - Checks if all characters are digits.
17. **isidentifier()** - Checks if the string is a valid Python identifier.
18. **islower()** - Checks if all characters are lowercase.
19. **isnumeric()** - Checks if all characters are numeric.
20. **isprintable()** - Checks if all characters are printable.
21. **isspace()** - Checks if all characters are whitespace.
22. **istitle()** - Checks if the string is title-cased.
23. **isupper()** - Checks if all characters are uppercase.
24. **join(iterable)** - Joins elements of an iterable with the string as a separator.
25. **ljust(width[, fillchar])** - Left-aligns the string with padding.

26. **lower()** - Converts all characters to lowercase.
27. **lstrip([chars])** - Removes leading characters (default is whitespace).
28. **maketrans(x[, y[, z]])** - Returns a translation table for `translate()`.
29. **partition(sep)** - Splits the string into three parts: before, separator, after.
30. **removeprefix(prefix)** - Removes a specified prefix (Python 3.9+).
31. **removesuffix(suffix)** - Removes a specified suffix (Python 3.9+).
32. **replace(old, new[, count])** - Replaces occurrences of a substring.
33. **rfind(sub[, start[, end]])** - Finds the last occurrence of a substring; returns -1 if not found.
34. **rindex(sub[, start[, end]])** - Finds the last occurrence of a substring; raises `ValueError` if not found.
35. **rjust(width[, fillchar])** - Right-aligns the string with padding.
36. **rpartition(sep)** - Splits the string into three parts, starting from the end.
37. **rsplit([sep[, maxsplit]])** - Splits the string from the right.
38. **rstrip([chars])** - Removes trailing characters (default is whitespace).
39. **split([sep[, maxsplit]])** - Splits the string into a list.
40. **splitlines([keepends])** - Splits the string at line breaks.
41. **startswith(prefix[, start[, end]])** - Checks if the string starts with the specified prefix.
42. **strip([chars])** - Removes leading and trailing characters (default is whitespace).
43. **swapcase()** - Swaps the case of all characters.
44. **title()** - Converts the string to title case.
45. **translate(table)** - Translates the string using a translation table.
46. **upper()** - Converts all characters to uppercase.
47. **zfill(width)** - Pads the string with zeros on the left.

Question 1: String Cleaning and Standardization

Scenario:

You are building a user data cleaning module for a form. You need to clean and standardize user inputs like names, email subjects, and comments.

Task:

Write a function `clean_and_standardize(text)` that performs the following operations:

1. **capitalize()**: Capitalize the first letter of the string.
 2. **casefold()**: Convert the string to lowercase (for aggressive standardization).
 3. **center()**: Center the text in a field of width 30, padded with *.
 4. **count()**: Count how many times the word "user" appears.
 5. **encode()**: Encode the text using UTF-8.
 6. **endswith()**: Check if the string ends with a period (.).
 7. **expandtabs()**: Replace tabs with spaces (tab size = 4).
 8. **find()**: Find the position of the first occurrence of the word "data".
 9. **format()**: Add "Input: {text}" using string formatting.
 10. **format_map()**: Use a dictionary to replace {text} with the original string.
-

Question 2: Content Filtering

Scenario:

You are designing a content moderation system to analyze and filter comments on a social media platform.

Task:

Write a function `filter_content(comment)` that processes a comment using these methods:

1. **index()**: Find the first occurrence of the word "spam".
 2. **isalnum()**: Check if the comment contains only alphanumeric characters.
 3. **isalpha()**: Check if all characters are alphabetic.
 4. **isascii()**: Verify if all characters are ASCII.
 5. **isdecimal()**: Check if the comment contains only decimal numbers.
 6. **isdigit()**: Verify if all characters are digits.
 7. **isidentifier()**: Check if the comment could be a valid Python identifier.
 8. **islower()**: Check if all characters are in lowercase.
 9. **isnumeric()**: Verify if the comment contains only numeric characters.
 10. **isprintable()**: Check if all characters are printable.
-

Question 3: Text Formatting for Reports

Scenario:

You need to format text for inclusion in a report, ensuring it is aligned and structured correctly.

Task:

Write a function `format_report(text)` that performs the following operations:

1. **isspace()**: Check if the text contains only whitespace.
 2. **istitle()**: Verify if the text is in title case.
 3. **isupper()**: Check if all characters are in uppercase.
 4. **join()**: Join a list of words into a single string, separated by commas.
 5. **ljust()**: Left-align the string with a width of 20.
 6. **lower()**: Convert the text to lowercase.
 7. **lstrip()**: Remove leading spaces or specific characters.
 8. **maketrans()**: Create a translation table for replacing "a" with "A".
 9. **partition()**: Split the string into three parts using the word "is" as a separator.
 10. **removeprefix()**: Remove the prefix "Report:" from the text.
-

Question 4: Word and Sentence Manipulation**Scenario:**

You are developing a tool to manipulate and analyze sentences provided by users.

Task:

Write a function `process_sentence(sentence)` that applies the following methods:

1. **removesuffix()**: Remove the suffix "Report" from the sentence.
2. **replace()**: Replace all occurrences of the word "old" with "new".
3. **rfind()**: Find the last occurrence of the word "end".
4. **rindex()**: Find the last occurrence of "important" and raise an error if not found.
5. **rjust()**: Right-align the text in a field of width 40.
6. **rpartition()**: Split the sentence into three parts, starting from the right.
7. **rsplit()**: Split the sentence into a list of words, starting from the right.
8. **rstrip()**: Remove trailing spaces or specific characters.
9. **split()**: Split the sentence into a list of words.
10. **splitlines()**: Split the sentence into a list of lines.

Question 5: String Transformations

Scenario:

You are tasked with transforming user-submitted data into uppercase, title case, and other formats for display on a dashboard.

Task:

Write a function `transform_data(data)` that applies the following methods:

1. **startswith()**: Check if the data starts with "User".
 2. **strip()**: Remove leading and trailing spaces.
 3. **swapcase()**: Swap the case of all characters in the string.
 4. **title()**: Convert the string to title case.
 5. **translate()**: Translate characters based on a given translation table (e.g., replace "e" with "3").
 6. **upper()**: Convert the string to uppercase.
 7. **zfill()**: Pad the string with zeros to make its length 10.
 8. **len()**: Calculate the length of the string.
 9. **ord()**: Find the Unicode code point of the first character.
 10. **chr()**: Convert a Unicode code point (e.g., 65) back to a character.
-